UNIDAD 2 — Redes Neuronales 1

Perceptrón Simple

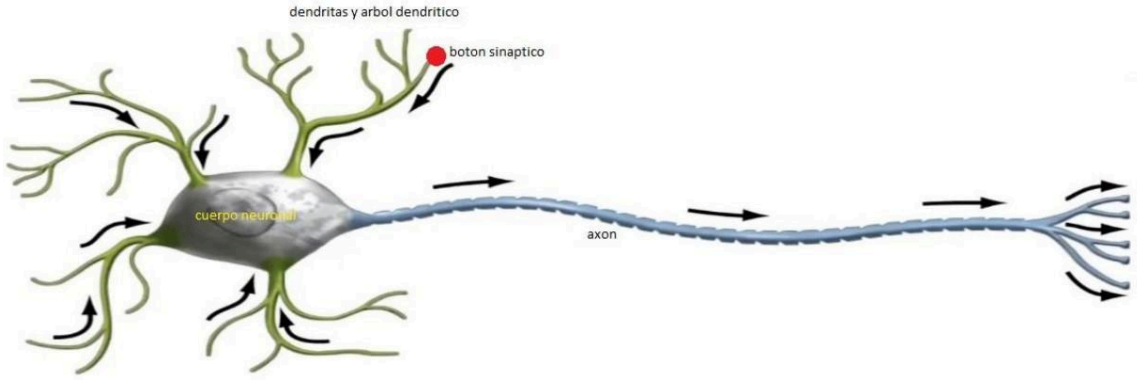
1- Fisiología básica de una neurona

Características de la neurona para poder modelar en una computadora:

- En el cerebro hay 10^11 neuronas, aunque actualmente hay modelos de esta cantidad no lo vamos a realizar.

- En las **redes neuronales**:
  - **N**o linealidad, un pequeño cambio en la entrada puede generar un gran cambio en la salida, o si dos entradas tiene dos salidas la suma de las entradas no es la suma de las salidas.
  - **P**aralelismo, hay muchas neuronas o unidades de proceso que convergen o concurren para resolver problemas complejos, pero cada unidad es muy sencilla o elemental.
  - **A**prendizaje, pueden aprender a partir de los datos, les damos ejemplos y ellas aprenden a resolver situaciones parecidas.
  - **G**eneralización, es la capacidad de resolver problemas nunca antes visto.
  - **A**daptabilidad, queremos que sean capaces de adaptarse a nuevas situaciones que no se vieron en el proceso de aprendizaje.
  - **R**obustez, la capacidad de funcionar bien aun cuando los datos tienen ruido o tiene faltantes.

Partes y fisiología de la neurona biológica:



- Cuerpo neuronal, que es donde se procesa la respuesta.
- Dendritas, que conforman un árbol dendrítico, es por donde ingresan los impulsos, las entradas.
- Axón, es por donde se comunica la respuesta, su salida.

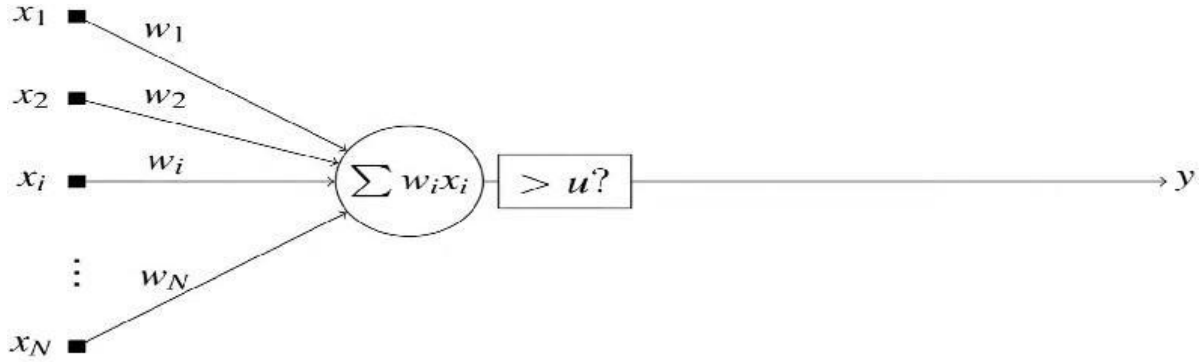
Comportamiento de una neurona:

- Botón sináptico, es donde se produce la sinapsis.
- **Sinapsis,** es la transmisión de los neurotransmisores.
  - Su finalidad es despolarizar la membrana de la neurona.
  - La membrana del núcleo tiene una polaridad, y cuando recibe una cierta cantidad de estímulos llega a despolarizarse y emite un único pulso por el axón.

    - Estos estímulos no son todos iguales, algunos son excitatorios y otros son inhibitorios, más o menos débil en cada caso, dependiendo del aprendizaje.
- Comportamiento todo o nada, solamente se envía el pulso cuando se logra la suficiente excitación de la neurona.
- Refuerzo de la sinapsis o aprendizaje, cuando se va aprendiendo se refuerzan los impulsos y los que eran débiles pasan a ser más fuertes y se va modificando la sinapsis a veces hasta se desconectan.

Modelo computacional de una neurona

- Modelo simplificado de la neurona o perceptrón simple 1:

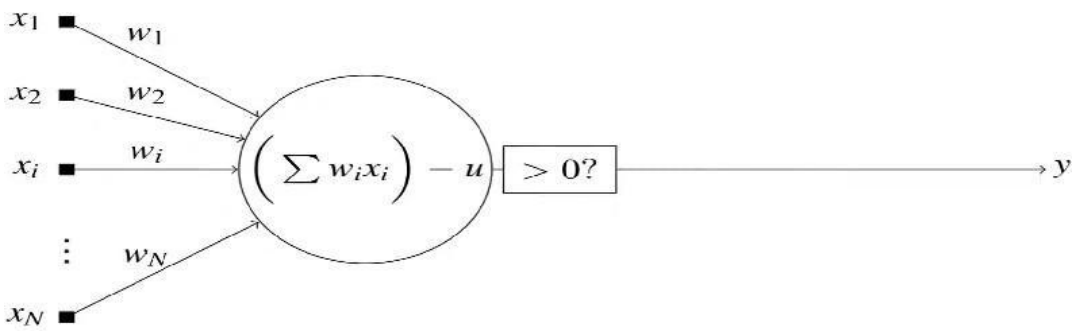


- Consideramos n entradas **xi = x1…xn** y cada una va a tener un peso sináptico **wi = w1…wn** que es un número real sin ninguna limitante. (si la sinapsis es excitatoria y grande, será un número alto, y viceversa)
  => a cada entrada **xi** se la pondera con el peso **wi** de acuerdo a la importancia que tengan para la despolarización de la neurona.

- En el cuerpo de la neurona:
    - Se hace una suma ponderada de las entradas con sus pesos.
    - Si la sumatoria es mayor al umbral definido **u**, se envía una respuesta o salida **y**.

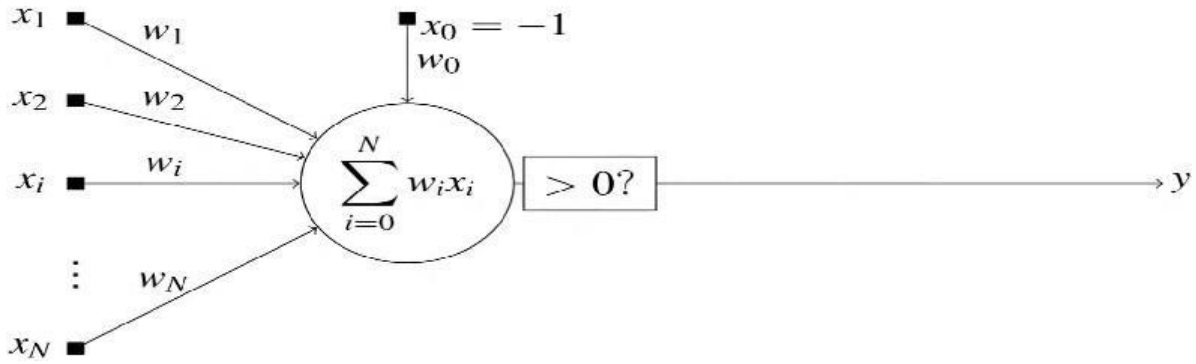
- Se realizan mayores simplificaciones asumimos que:
    - Las entradas **xi**, llegan simultáneamente.
    - Se evalúa de manera instantánea en el cuerpo neuronal si se supera el umbral u.

- Modelo simplificado de la neurona o perceptrón simple 2:



- Es una ecuación más sencilla, introduciendo el umbral como resta en el núcleo de la neurona.

- Modelo simplificado de la neurona o perceptrón simple 3:



- Una última simplificación, en la cual hacemos que el umbral **u** sea la multiplicación de una entrada **x0,** que siempre es **-1**, por un peso **w0**. Para incluirla, cambiamos el índice de la sumatoria, de 1 a cero, entonces estamos incluyendo la nueva entrada.

=> llegamos a lo que es un Perceptrón Simple:

Perceptrón simple:

- Es un producto interno entre las entradas y peso sináptico, denominado activación lineal **v**, y un umbral:

$$y = \phi(v - u) = \phi\left(\sum_{i=1}^{N} w_i x_i - u\right)$$

y con la entrada extendida: **x0 = -1, w0 = u:**

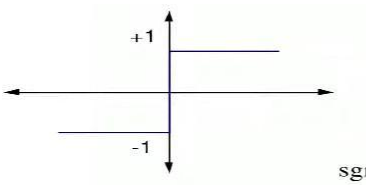
$$y = \phi\left(\sum_{i=0}^{N} w_i x_i\right) = \phi\left(\langle \mathbf{w}, \mathbf{x} \rangle\right)$$

Función de Activación:

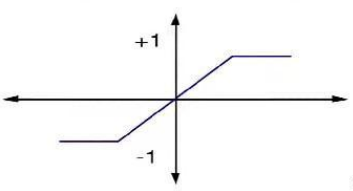
$$\emptyset(z) = y - \emptyset\left(\sum_{i=0}^{N} w_i\, x_i\right) = \emptyset(< w, x >)$$

?

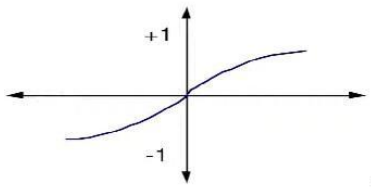
Podemos obtener muchas funciones de activación:



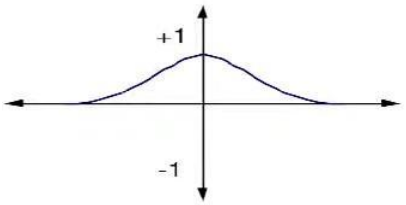
$$\text{sgn}(z) = \begin{cases} -1 & \text{si } z < 0 \\ +1 & \text{si } z > 0 \end{cases}$$

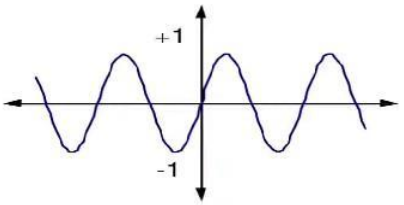


$$\text{sln}(z) = \begin{cases} -1 & \text{si } z < -a \\ \alpha z & \text{si } -a < z < 0 \\ +1 & \text{si } z \geq a \end{cases}$$

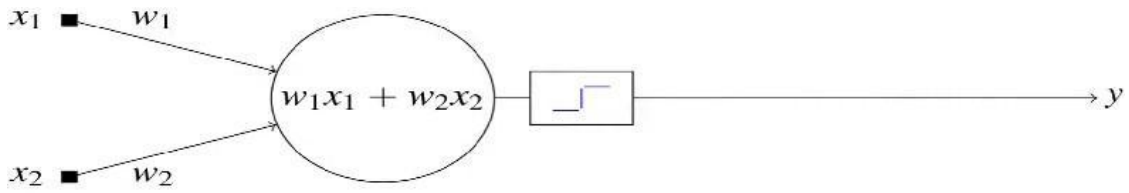


$$\text{sig}(z) = \frac{1 - e^{-az}}{1 + e^{-az}}$$





Un perceptrón simple con 2 entradas, y fda signo:

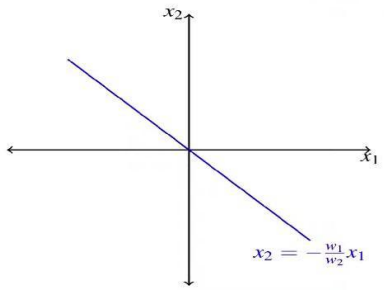


$$y = \text{sgn}(w_1x_1 + w_2x_2)$$

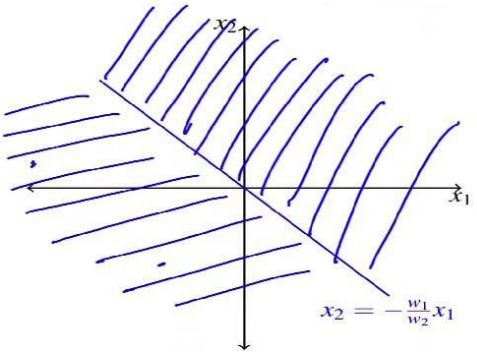
$w_1x_1 + w_2x_2 > 0?$ → <u>punto crítico de funcionamiento de la neurona:</u>

- <u>Representación gráfica</u> de la zona donde la suma ponderada da cero:

$w_1x_1 + w_2x_2 = 0$ → ecuación de una recta →

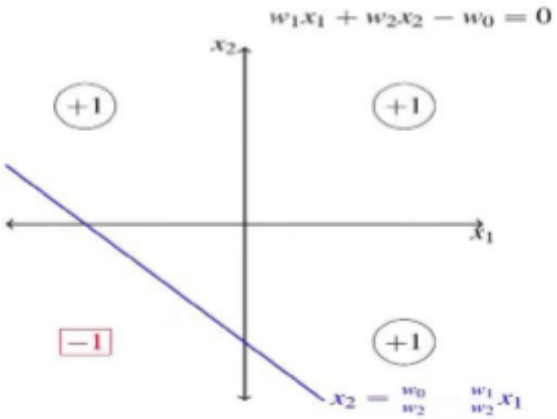


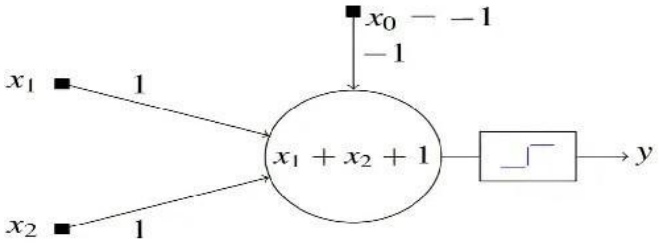
→ lo que nos delimita 2 zonas de comportamiento:



- Necesidad del bias/sesgo:

    - Cuando necesitamos que la recta NO pase por el orígen para delimitar las 2 zonas de comportamiento, agregamos el umbral **w0** a la ecuación:
    - Por ejemplo, en el comportamiento del OR:





Acá le pusimos nosotros los pesos sinápticos, pero la idea es que la neurona aprenda sola a asignar los pesos…

Algoritmos de aprendizaje: ¿Cómo aprende solo?

Entrenamiento por corrección de error:

1. Inicialización al azar
2. Se muestran muchos ejemplos con las salidas esperadas $y_d(n)$ y en cada caso se calcula:

$y(n) = \phi(\langle \mathbf{w}(n), \mathbf{x}(n) \rangle)$ → incorporamos la **n** (la iteración en el proceso de aprendizaje, cada vez que le muestro un ejemplo):

- SI la salida de la red es **correcta:** $y(n) = y_d(n)$
  no se hacen cambios: *principio de mínima perturbación.*
- SI la salida de la red es **incorrecta:** $y(n) \neq y_d(n)$
  penalización: se *actualizan los pesos en el sentido opuesto* al cual con el que contribuyeron a la salida incorrecta:

  - **SI** $y(n) = +1$ **mientras** $y_d(n) = -1$
    $\Rightarrow \mathbf{w}(n+1) = \mathbf{w}(n) - \eta\mathbf{x}(n)$

  => *restamos una proporción de las entradas que entraron.*

  - **SI** $y(n) = -1$ **mientras** $y_d(n) = +1$
    $\Rightarrow \mathbf{w}(n+1) = \mathbf{w}(n) + \eta\mathbf{x}(n)$

  => *sumamos una proporción de las entradas que entraron*

O mejor, para unir todos los casos (correcto e incorrecto):

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\eta}{2}[y_d(n) - y(n)]\mathbf{x}(n)$$

=> *al peso actual le sumamos* **una proporción*(la diferencia entre la salida deseada y la salida de la neurona)*(entrada)**

(el ½, se pone para simplificar la dif, que puede dar +2 o -2)

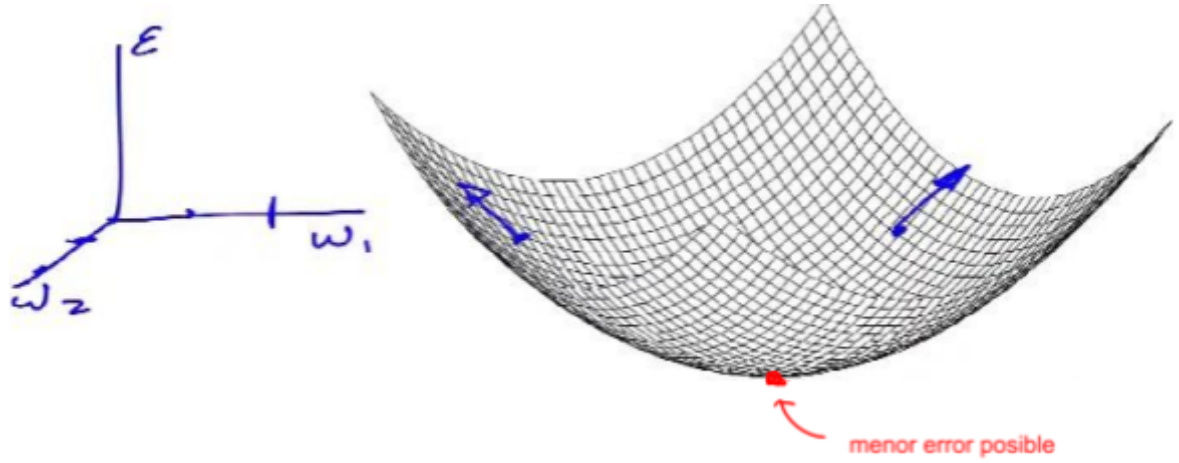
3. Volver a 2 hasta satisfacer algún criterio de finalización.

Entrenamiento por <u>métodos de gradiente</u>: perceptrón simple

→ Forma de encontrar los pesos sinápticos con un mayor sustento matemático

→ **Concepto**: Mover los pesos iteración a iteración en la dirección en que se reduce el error, <u>dirección que es opuesta a su gradiente con respecto a los pesos</u>

→ **Interpretación gráfica:**



→ **Ecuación básica:**

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \mu\nabla_w\xi(\mathbf{w}(n))$$

pesos en la it. sig. **=** peso actual - vel. de aprendizaje*gradiente de error respecto a los pesos

→ <u>la vel. de aprendizaje hace que nos movamos más o menos rápido por la superficie</u>

→ si la sup. es suave puedo ir más rápido, y viceversa.

→ **Aplicación:**

- Caso sencillo: perceptrón simple (least mean squares)
- **Caso general: perceptrón multicapa (back-propagation)**

Perceptrón simple: método de gradiente (caso lineal)

- Medimos el error con el criterio de error instantáneo (d(n) = salida deseada)

$$e^2(n) = [d(n) - y(n)]^2 = [d(n) - \langle \mathbf{w}(n), \mathbf{x}(n) \rangle]^2$$

- Caso lineal: **eliminamos** la función de activación, es una recta.

- Para calcular el gradiente, tenemos que derivar respecto a los pesos:

$$\nabla_w e^2(n) = 2\,[d(n) - \langle \mathbf{w}(n), \mathbf{x}(n) \rangle]\,(-\mathbf{x}(n))$$

$$\nabla_w e^2(n) = 2e(n)(-\mathbf{x}(n))$$

reemplazando en: $\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \nabla_w \xi(\mathbf{w}(n))$

$$\boxed{\mathbf{w}(n+1) = \mathbf{w}(n) + 2\mu e(n)\mathbf{x}(n)}$$

→ lo único que cambió respecto al método anterior es que tenemos 2μ, donde antes teníamos η.

Ejemplo:

Supongamos que: $w_0 = +1 \quad x_0 = -1 \quad d = -1$
$w_1 = +1 \quad x_1 = +1$
$w_2 = +1 \quad x_2 = +1$

$$y = \text{sgn}(-1 + 1 + 1) \rightarrow 1$$

$$\mathbf{w}(n+1) = \mathbf{w}(n) + 2\mu e(n)\mathbf{x}(n)$$

$$\mathbf{w}(n+1) = \begin{bmatrix} +1 \\ +1 \\ +1 \end{bmatrix} + 2\mu(-1-1) \begin{bmatrix} -1 \\ +1 \\ +1 \end{bmatrix}$$

suponiendo que $\mu = \frac{1}{2}$

$$\mathbf{w}(n+1) = \begin{bmatrix} +1 \\ +1 \\ +1 \end{bmatrix} + \begin{bmatrix} +2 \\ -2 \\ -2 \end{bmatrix} = \begin{bmatrix} +3 \\ -1 \\ -1 \end{bmatrix}$$

Qué sucede si volvemos a poner: $\mathbf{x}(n+1) = \begin{bmatrix} -1 \\ +1 \\ +1 \end{bmatrix}$?

$$y = \text{sgn}(-3 - 1 - 1) \rightarrow -1$$

$$\boxed{\text{ahora } e(n+1) = -1 - (-1) = 0}$$

•